NTNU

Kunnskap for en bedre verden

NTNU

TDT4145 Datamodellering og databasesystemer

Prosjektinnlevering del 1

TreningsDB

Forfatter(e):
Prosjektgruppe

Merknad:
SQL-skript er ikke gjengitt i rapporten; besvarelsen refererer til `schema.sql`.

8. mars 2026

Innhold

Figurliste **i**

Tabelliste **i**

1 Innledning **1**

2 ER-modell **1**

3 ER-modell og forutsetninger **1**

4 Relasjonsdatabaseskjema og normalformer **2**

5 c) SQL-skript og applikasjonslogikk **5**

6 d) Svar på spørsmål **6**

7 Bruk av kunstig intelligens **7**

Figurliste

1 ER-modell for TreningsDB. 1

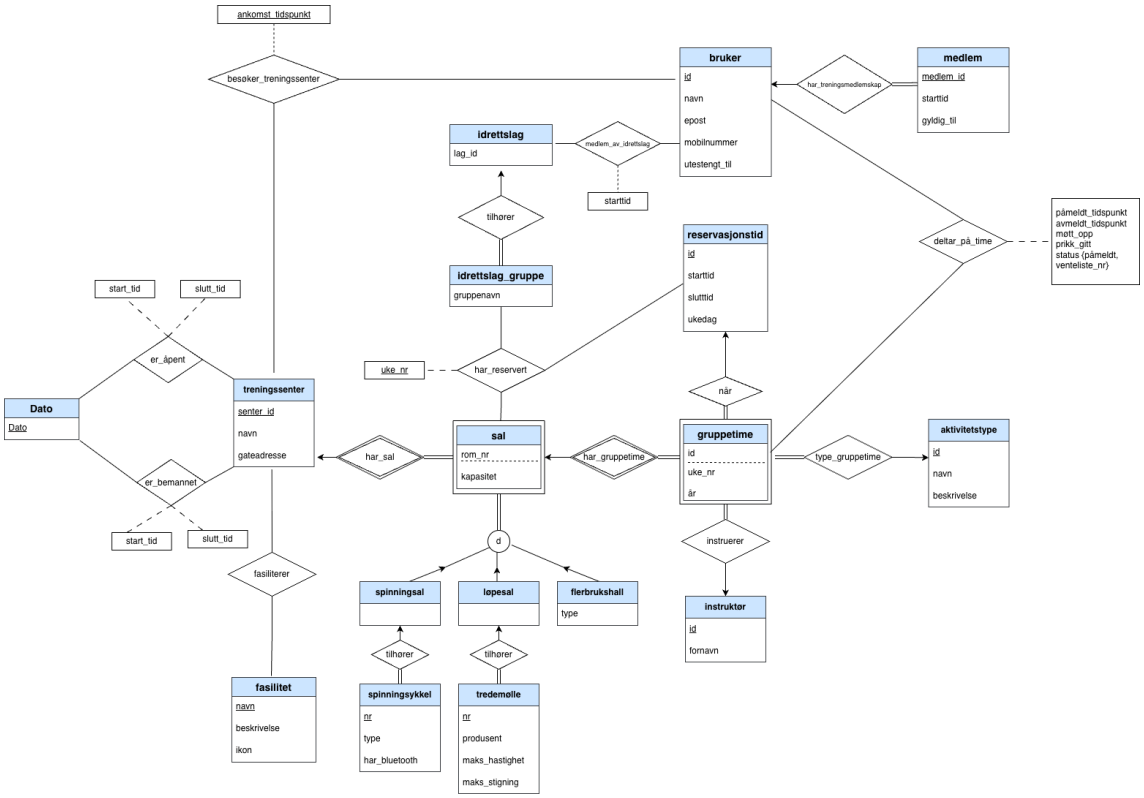
Tabelliste

1 Innledning

Denne rapporten er prosjektinnlevering del 1 i TDT4145 Datamodellering og databasesystemer. I denne delen av prosjektet er oppgaven å utvikle en ER-modell for domenet, oversette modellen til et relasjonsdatabaseskjema, beskrive normalformer, samt dokumentere hvilke restriksjoner som må håndteres utenfor selve skjemaet. Rapporten er derfor strukturert i fire deler som svarer til oppgavens punkter a, b, c og d.

2 ER-modell

Figur 1 viser ER-modellen som inngår i besvarelsen.



Figur 1: ER-modell for TreningsDB.

3 ER-modell og forutsetninger

I denne delen presenterer vi ER-modellen for systemet, samt forutsetningene vi har lagt til grunn og restriksjoner som ikke kan uttrykkes fullt ut i ER-/EER-modellen.

Forutsetninger

Feltet deltar_på_time.prikk_dato settes av instruktør/ansvarlig for timen. Dette kan i praksis gjøres ved at man scanner seg inn i timen. Grunnen til at vi har løst det slik, er at det gir fleksibilitet for når en prikk skal settes. Blant annet kan man ha møtt opp fem minutter før timen, men ikke rukket å scanne seg inn grunnet kø. Når en bruker deltar på en gruppereservasjon, har man fasilitert for at man kan sjekke om brukeren er en del av idrettslaget. Vi ser derimot ikke nødvendigheten av at man skal ha egen påmeldings-logikk for påmelding av brukere for hver enkelt gruppereservasjon.

En instruktør-bruker kan ikke melde seg på timer. Kommunikasjon med instruktører går via et eksternt system. Dersom en instruktør vil trene, må den lage en brukerkonto og melde seg på som bruker. Dette er for å fasilitere for at instruktører kan holde timer uten å nødvendigvis være med i SiT Dersom man får tre prikker og blir utestengt, så avlyses alle fremtidige bookinger ved at man fjerner brukeren fra deltar_på_time tabellen. Det er 168 (24 * 7) ulike tidsblokker; hver tidsblokk starter kl.xx:00, og varer nøyaktig én time. Dersom en gruppe ønsker å booke en hall i flere timer, vil dette gjøres gjennom flere bookinger. Idrettslaggrupper sine navn er entydig definert. For eksempel vil NTNUI sitt 2. lag i fotball hete noe slikt som "NTNUI 2.lag fotball", og ikke noe generisk som "fotballlag". Vi tillater to ulike senterbesøk gjort av samme bruker, på samme senter, til samme tid. Dette er siden vi kan bruke disse "duplikatene" til å detektere mistenkelig oppførsel av brukere. Eksempelvis at man scanner inn en annen person uten medlemskap. "gruppereservasjon" og "gruppetime"- tabellene resettes hvert år. Da kan denne dataen eksempelvis arkiveres et annet sted, slik at tabellene ikke blir for store. Regler for prikk, for sen avmelding og uteblivelse må håndteres i applikasjonen. Schemaet lagrer bare resultatet (prikk_dato), ikke logikken.

Restriksjoner som ikke uttrykkes gjennom EER-modellen

Et senter kan kun være bemannet dersom det også er åpent. En bruker, og en instruktør kan begge kun være med på én gruppetime per timeslot. Dette beskrives i detalj i oppgave d. En bruker skal bare kunne meldes på gruppetimer dersom brukeren ikke er utestengt. Overlapp må kontrolleres i applikasjonen; samme sal kan ikke ha både gruppetime og gruppereservasjon i samme relevante tidsrom Senterbesøk, gruppetimer og gruppereservasjoner bør bare opprettes innenfor senterets åpningstid og eventuelle bemanningskrav. Dette må valideres i applikasjonen.

4 Relasjonsdatabaseskjema og normalformer

I denne delen presenterer vi ER-modellen oversatt til relasjonsdatabaseskjema. For hver tabell oppgir vi nøkler, fremmednøkler og en kort begrunnelse for hvorfor tabellen er på BCNF.

Relasjonsskjema 2.0

bruker(*id*, navn, epost, mobilnr, utestengt_til)

BCNF: Alle attributter er kun funksjonelt avhengige av primærnøkkelen id.

medlem(*id*, starttid, gyldig_til, bruker_id)

Fremmednøkkel: bruker_id → bruker(id)
BCNF: Alle attributter avhenger kun av id.

medlem_av(*bruker_id*, *idrettslag_id*, starttid)

Fremmednøkler:

- bruker_id → bruker(id)
- idrettslag_id → idrettslag(id)

BCNF: Sammensatt primærnøkkel der starttid avhenger av hele nøkkelen.

idrettslag(*id*, navn)

BCNF: Alle attributter er funksjonelt avhengige av primærnøkkelen id.

idrettslag_gruppe(*gruppenavn*, idrettslag_id)

Fremmednøkkel: idrettslag_id → idrettslag(id)
BCNF: idrettslag_id er avhengig av gruppenavn. Refererer til forutsetningene beskrevet i deloppgave a.

gruppereservasjon(*id*, uke_nr, gruppenavn, tidsblokk_starttid, tidsblokk_ukedag, senter_id, sal_nr)

Fremmednøkler:

- (gruppenavn, idrettslag_id) → idrettslag_gruppe(gruppenavn, idrettslag_id)
- (tidsblokk_starttid, tidsblokk_ukedag) → tidsblokk(starttid, ukedag)
- (senter_id, sal_nr) → sal(senter_id, nr)

BCNF: Alle attributter avhenger kun av id.

tidsblokk(*starttid*, *ukedag*)

BCNF: Sammensatt primærnøkkel, ingen andre attributter.

gruppetime(*id*, uke_nr, år, instruktør_id, aktivitetstype, senter_id, sal_nr)

Fremmednøkler:

- instruktør_id → instruktør(id)
- aktivitetstype → aktivitetstype(navn)
- (senter_id, sal_nr) → sal(senter_id, nr)

BCNF: Alle attributter avhenger kun av id.

senterbesøk(*id*, ankomsttidspunkt, bruker_id, senter_id)

Fremmednøkler:

- bruker_id → bruker(id)
- senter_id → senter(id)

BCNF: Alle attributter avhenger kun av id.

senter(*id*, navn, gateadresse)

BCNF: Alle attributter avhenger kun av id.

senteråpningstid(*dato*, *senter_id*, start_tid, slutt_tid)

Fremmednøkkel: senter_id → senter(id)
BCNF: Start_tid og slutt_tid avhenger kun av hele den sammensatte nøkkelen. Dette er den eneste funksjonelle avhengigheten vi har.

senterbemanning(*dato*, *senter_id*, start_tid, slutt_tid)

Fremmednøkkel: senter_id → senter(id)
BCNF: Start_tid og slutt_tid avhenger kun av hele den sammensatte nøkkelen.

fasilitet(*navn*, ikon)

BCNF: Ikon avhenger kun av navn som er primærnøkkel.

sal(*senter_id*, *nr*, kapasitet)

Fremmednøkkel: senter_id → senter(id)
BCNF: Kapasitet er avhengig av hele primærnøkkelen.

spinningsal(*senter_id*, *sal_nr*)

Fremmednøkkel: (senter_id, sal_nr) → sal(senter_id, nr)
BCNF: Kun primærnøkkel, ingen andre attributter.

løpesal(*senter_id*, *sal_nr*)

Fremmednøkkel: (senter_id, sal_nr) → sal(senter_id, nr)
BCNF: Kun primærnøkkel, ingen andre attributter.

flerbrukshall(*senter_id*, *sal_nr*, type)

Fremmednøkkel: (senter_id, sal_nr) → sal(senter_id, nr)
BCNF: Type avhenger av både senter_id og sal_nr, derfor på BCNF.

spinningsykkel(*id*, type, har_bluetooth, senter_id, sal_nr)

Fremmednøkkel: (senter_id, sal_nr) → spinningsal(senter_id, sal_nr)
BCNF: Alle attributter er kun avhengige av id.

tredemølle(*id*, produsent, maks_hastighet, maks_stigning, senter_id, sal_nr) – CHECK((senter_id, sal_nr in løpesal)

Fremmednøkkel: (senter_id, sal_nr) → løpesal(senter_id, sal_nr)
BCNF: Alle attributter avhenger kun av id.

time_skjer_i(*gruppetime_id*, tidsblokk_starttid, tidsblokk_ukedag)

Fremmednøkler:

- gruppetime_id → gruppetime(id)
- (tidsblokk_starttid, tidsblokk_ukedag) → tidsblokk(starttid, ukedag)

BCNF: Alle attributter avhenger kun av gruppetime_id.

deltar_på_time(*gruppetime_id*, *bruker_id*, påmeldt_tidspunkt, avmeldt_tidspunkt, prikk_dato)

Fremmednøkler:

- gruppetime_id → gruppetime(id)
- bruker_id → bruker(id)

BCNF: Sammensatt nøkkel utgjør venstresiden av alle funksjonelle avhengigheter.

fasiliterer(*senter_id*, *fasilitetsnavn*)

Fremmednøkler:

- senter_id → senter(id)
- fasilitetsnavn → fasilitet(navn)

BCNF: Kun sammensatt primærnøkkel, ingen andre attributter.

instruktør(*id*, fornavn)

BCNF: Fornavn avhenger av id.

aktivitetstype(*navn*, beskrivelse)

BCNF: Beskrivelse avhenger kun av navnet som er primærnøkkel.

5 c) SQL-skript og applikasjonslogikk

c) Et SQL-script som konstruerer databasen med tabellene. Husk å spesifiser primær- og fremmednøkler, samt andre nødvendige restriksjoner. Dokumenter restriksjoner som ikke uttrykkes i relasjonsdatabaseskjemaet og derfor må håndteres i applikasjonsprogrammene.

Det er to restriksjoner som risikerer å gå tapt i oversettelsen fra ER-diagram til relasjonsdatabaseskjemaet.

Den ene er kardinalitetskrav. En mange-til-en-relasjon der deltakelsen på mange-siden er total, løses ved å legge primærnøkkelen fra en-siden inn som en fremmednøkkel i tabellen på mange-siden, og definere denne som NOT NULL. Dette sikrer både referanseintegritet og kravet om total deltakelse.

Den andre er at skjemaet ikke sikrer at hver entitet i relasjonen sal forekommer i nøyaktig én av relasjonene spinningsal, løpesal eller flerbrukssal. Dette må derfor kontrolleres i applikasjonslaget.

SQL: refererer til schema.sql

6 d) Svar på spørsmål

d) Vi ønsker svar på følgende spørsmål:

1. Hvordan (eller kan) dere sørge for at en instruktør ikke kan være til stede på to forskjellige plasser samtidig? Kan dere få til dette ved hjelp av modellen deres, eller må noe slikt implementeres i programvaren.
2. Det samme spørsmålet for en bruker. Kan dette løses i modellen eller må det gjøres i programvaren?

svar Vi tar i bruk samme løsning for både problemstilling én og to. Vi mener at det er mulig, men ikke optimalt å løse dette ved direkte implementasjon i ER-diagrammet. Vi ønsker likevel å forklare hvordan vi kan løse problemet. Man kan opprette to nye entiteter, "instruktør_tid" og "bruker_tid". Disse entitetene kan bestå av instruktør/bruker-id, og en reservasjonstid-id. Videre kan man legge inn en én til én relasjon mellom "gruppetime" og "instruktør_tid". Dette vil sørge for at for hvert tidsintervall så vil én instruktør aldri bli koblet til to gruppetimer. Ekvivalent kan vi legge til en én til én relasjon mellom "gruppetime" og "bruker_tid", med identisk funksjonalitet og implementasjon som beskrevet. Det er flere grunner til at vi likevel ikke velger å løse oppgaven på denne måten. Ett problem som dukker opp med denne implementasjonen er at gruppetimer kun kan kobles til én instruktør-tid. Dette medfører at alle gruppetimer må ha lik lengde, siden vi antar at alle tidsintervall er av lik lengde. Et annet, og desto mer overveiende problem, er at vi vurderer informasjonen lagret i disse to nye entitetene som overflødig informasjon. Før man kobler en instruktør eller bruker til en gruppetime kan man enkelt sjekke om det finnes en annen gruppetime der den brukeren eller instruktøren allerede er meldt på i det gitte tidsintervallet. Dette kan blant annet implementeres ved hjelp av en "integrity constraint", der man før innsetting sjekker "gruppetime"-tabellen for å se om det finnes en annen gruppetime med samsvarende instruktør og tidsintervall.

1. Fra hvilket usecase testes det for / opprettes utestengelse (svartelisting) ? Det trenger ikke være en av de oppgitte usecasene.

Svartelisting opprettes naturlig i usecase 3, da det er her det registreres oppmøte. Systemet setter prikk_dato på en deltaker som ikke møtte opp. Etter at prikken er registrert, bør da systemet automatisk sjekke om brukeren nå har 3 eller flere prikker innen de siste 30 dagene. Hvis brukeren tilfredsstiller kravet til utestengelse, bør man sette utestengt_til = NOW() + 30 dager på brukeren.

Usecase 6 er selve implementasjonen av svartelistings-logikken, men det er usecase 3 som kaller denne logikken ved å registrere prikk etter manglende oppmøte. Dette kan implementeres ved bruk av en trigger.

1. Statistikk er det noe som må lagres eksplisitt eller går det an å bare gjøre queries for å få svar? Diskuter dette.

Statistikk er noe vi mener ikke må lagres eksplisitt, da det er derived data. I de fleste tilfeller holder det med queries. Ønsker man for eksempel å finne hvem som trener mest, kan man bruke count gruppert på bruker.

Vi ønsker likevel å trekke fram to tilfeller der det kan være lurt med eksplisitt lagring. For det første, hvis systemet skal håndtere større datamengder over tid, kan det lønne seg å lagre aggregert

statistikk i egne tabeller fremfor å beregne nytt hver gang. For det andre nevner oppgaven eksplisitt at SiT ønsker statistikk for å planlegge for nye semestre. Det motiverer for å lagre statistikk eksplisitt, hvis man ønsker bedre historiske sammenligningsgrunnlag, La oss si at SiT ønsker å bygge om på saler. Man trenger da historikk over tid, og da kan historiske queries gi feil oppfyllingsgrad for tidligere økter. Det er fordi om kapasiteten til salen endres, vil kapasiteten på tidspunktet for økten gå tapt.

7 Bruk av kunstig intelligens

Bruk av kunstlig intelligens Vi har brukt kunstig intelligens for å verifisere at ER-diagrammet vårt er korrekt. Vi brukte en god del tid på selve designet, og ønsket å verifisere om prinsippene vi la til grunn for designet var korrekte.

Vi har også brukt kunstig intelligens som sparringspartner for å oppdage feil i sql-skjemaet, og for å konvertere rapporten til latex.